

Counter Operation Programs

13) Write an ALP for Decimal Up-Counter.

```
ORG 0000H      ; Initialize the memory
MOV A, #00H    ; Load accumulator with 00H
BACK: MOV P0, A ; Accumulator content moved to Port 0
        ACALL DELAY ; Call delay subroutine program
        ADD A, #01H ; A=A+1
        DA A        ; Decimal adjust accumulator
        SJMP BACK   ; Short jump to the label BACK
DELAY: MOV R1, #0FH ; Load R1 with 0FH
L1      : MOV R2, #0FFH ; Load R2 with 0FFH
L2      : MOV R3, #0FFH ; Load R3 with 0FFH
L3      : DJNZ R3, L3    ; Decrement R3 and jump to L3 if R3≠0
        DJNZ R2, L2    ; Decrement R2 and jump to L2 if R2≠0
        DJNZ R1, L1    ; Decrement R1 and jump to L1 if R1≠0
        RET           ; Return from subroutine
END            ; end of program
```

14) Write an ALP for Decimal Down-Counter.

```
ORG 0000H      ; Initialize the memory

MOV A, #99H    ; Load accumulator with 99H

BACK: MOV P0, A ; Accumulator content is moved to Port 0

ACALL DELAY    ; Call delay subroutine program

ADD A, #99H    ; A=A+99      99+99=198

DA A           ; Decimal adjust accumulator

SJMP BACK      ; Short jump to the label BACK

DELAY: MOV R1, #0FH ; Load R1 with 0FH

L1 : MOV R2, #0FFH ; Load R2 with 0FFH

L2 : MOV R3, #0FFH ; Load R3 with 0FFH

L3 : DJNZ R3, L3    ; Decrement R3 and jump to L3 if R3≠0

      DJNZ R2, L2    ; Decrement R2 and jump to L2 if R2≠0

      DJNZ R1, L1    ; Decrement R1 and jump to L1 if R1≠0

      RET           ; Return from subroutine

      END           ; end of program
```

15) Write an ALP for Hexadecimal Up-Counter.

```
ORG 0000H      ; Initialize the memory

MOV A, #00H    ; Load accumulator with 00H

BACK: MOV P0, A ; Accumulator content moved to Port 0

ACALL DELAY    ; Call delay subroutine program

ADD A, #01H    ; A=A+1

SJMP BACK      ; Short jump to the label BACK

DELAY: MOV R1, #0FH ; Load R1 with 0FH

L1 : MOV R2, #0FFH ; Load R2 with 0FFH

L2 : MOV R3, #0FFH ; Load R3 with 0FFH

L3 : DJNZ R3, L3    ; Decrement R3 and jump to L3 if R3≠0
      DJNZ R2, L2    ; Decrement R2 and jump to L2 if R2≠0
      DJNZ R1, L1    ; Decrement R1 and jump to L1 if R1≠0

RET            ; Return from subroutine

END            ; end of program
```

16) Write an ALP for Hexadecimal Down-Counter.

```
ORG 0000H      ; Initialize the memory

MOV A, #0FFH   ; Load accumulator with FFH

BACK: MOV P0, A ; Accumulator content moved to Port 0

ACALL DELAY    ; Call delay subroutine program

ADD A, #0FFH   ; A=A+FF

SJMP BACK      ; Short jump to the label BACK

DELAY: MOV R1, #0FH ; Load R1 with 0FH

L1 : MOV R2, #0FFH ; Load R2 with 0FFH

L2 : MOV R3, #0FFH ; Load R3 with 0FFH

L3 : DJNZ R3, L3    ; Decrement R3 and jump to L3 if R3≠0
      DJNZ R2, L2    ; Decrement R2 and jump to L2 if R2≠0
      DJNZ R1, L1    ; Decrement R1 and jump to L1 if R1≠0

RET            ; Return from subroutine

END            ; end of program
```

Additional Counter Programs

1) Aim: To write an ALP to implement a binary up counter.

Label	Mnemonic	Operands	Comment
	ORG	0000H	;Initialize the memory to 0000H
	MOV	A, #00H	;Load the accumulator with 00H
BACK:	MOV	P0, A	;Copy the contents of accumulator to ;Port 0
	ACALL	DELAY	;Call DELAY subroutine
	ADD	A, #01H	;Add 01H to the contents of accumulator
	SJMP	BACK	;Jump to BACK
DELAY:	MOV	R0, #0FH	;Load R0 with 0FH
LOOP2:	MOV	R1, #0FFH	;Load R1 with FFH
LOOP1:	MOV	R2, #0FFH	;Load R2 with FFH
LOOP:	DJNZ	R2, LOOP	;Decrement R2 and jump to LOOP if R2≠0
	DJNZ	R1, LOOP1	;Decrement R1 and jump to LOOP1 if R1≠0
	DJNZ	R0, LOOP2	;Decrement R0 and jump to LOOP2 if R0≠0
	RET		;Return from subroutine
	END		;End of the program

2) Aim: To write an ALP to implement a binary down counter.

Label	Mnemonic	Operands	Comment
	ORG	0000H	;Initialize the memory to 0000H
	MOV	A, #00H	;Load the accumulator with 00H
BACK:	MOV	P0, A	;Copy the contents of accumulator to ;Port 0
	ACALL	DELAY	;Call DELAY subroutine
	SUBB	A, #01H	;Subtract 01H to the contents of ;accumulator
	SJMP	BACK	;Jump to BACK
DELAY:	MOV	R0, #0FH	;Load R0 with 0FH
LOOP2:	MOV	R1, #0FFH	;Load R1 with FFH
LOOP1:	MOV	R2, #0FFH	;Load R2 with FFH
LOOP:	DJNZ	R2, LOOP	;Decrement R2 and jump to LOOP if R2≠0
	DJNZ	R1, LOOP1	;Decrement R1 and jump to LOOP1 if R1≠0
	DJNZ	R0, LOOP2	;Decrement R0 and jump to LOOP2 if R0≠0
	RET		;Return from subroutine
	END		;End of the program

3) Aim: To write an ALP to implement a BCD up counter.

Label	Mnemonic	Operands	Comment
	ORG	0000H	;Initialize the memory to 0000H
	MOV	A,#00H	;Load the accumulator with 00H
BACK:	MOV	P0,A	;Copy the contents of accumulator to ;Port 0
	ACALL	DELAY	;Call DELAY subroutine
	ADD	A,#01H	;Add 01H to the contents of accumulator
	DA	A	;Decimal adjust accumulator
	SJMP	BACK	;Jump to BACK
DELAY:	MOV	R0,#0FH	;Load R0 with 0FH
LOOP2:	MOV	R1,#0FFH	;Load R1 with FFH
LOOP1:	MOV	R2,#0FFH	;Load R2 with FFH
LOOP:	DJNZ	R2,LOOP	;Decrement R2 and jump to LOOP if R2≠0
	DJNZ	R1,LOOP1	;Decrement R1 and jump to LOOP1 if R1≠0
	DJNZ	R0,LOOP2	;Decrement R0 and jump to LOOP2 if R0≠0
	RET		;Return from subroutine
	END		;End of the program

4) Aim: To write an ALP to implement a BCD down counter.

Label	Mnemonic	Operands	Comment
	ORG	0000H	;Initialize the memory to 0000H
	MOV	A,#00H	;Load the accumulator with 00H
BACK:	MOV	P0,A	;Copy the contents of accumulator to ;Port 0
	ACALL	DELAY	;Call DELAY subroutine
	ADD	A,#99H	;Add 99H to the contents of accumulator
	DA	A	;Decimal adjust accumulator
	SJMP	BACK	;Jump to BACK
DELAY:	MOV	R0,#0FH	;Load R0 with 0FH
LOOP2:	MOV	R1,#0FFH	;Load R1 with FFH
LOOP1:	MOV	R2,#0FFH	;Load R2 with FFH
LOOP:	DJNZ	R2,LOOP	;Decrement R2 and jump to LOOP if R2≠0
	DJNZ	R1,LOOP1	;Decrement R1 and jump to LOOP1 if R1≠0
	DJNZ	R0,LOOP2	;Decrement R0 and jump to LOOP2 if R0≠0
	RET		;Return from subroutine
	END		;End of the program